Lab Report: Embedded Programming in C (Lab 1)

Gagan Adhikari

November 23, 2025

Objectives

This report presents the answers to the questions from Lab 1: Getting Started with Embedded Programming in C using the ATmega328P microcontroller.

Questions and Answers

Q1. What is the purpose of flash memory? What is the purpose of data memory?

Flash memory is non-volatile and stores the program code (firmware). It retains its contents even when power is off. Data memory (SRAM) is volatile and used during program execution to store variables, stack, and temporary data.

Q2. How many bytes are used to represent an `int` value in ATmega328P?

On the ATmega328P, an `int` is represented using 2 bytes (16 bits).



Figure 1: size of int in bytes

Q3. It is important to know how an integer is stored in memory. It is good to know in which order the value is stored in memory. In little-endia, the least significant byte (LSB) is stored at the lowest memory address, and the most significant byte (MSB) is stored at the highest memory address. In big-endian, the most significant byte (MSB) is stored at the lowest memory address, and the least significant byte (LSB) is stored at the highest address. Does ATmega 328p use little-endian or big-endian for storing an int value? How do you find out?

The ATmega328P uses little-endian format. This was observed when variable `x = 0x1234` was stored as `34` at the lower memory address and `12` at the higher memory address.

Figure 2: showing memory address

Q4. When the statement `ch2 = ch1 + 2` was executed, why did `ch2` have a value of 50?

`ch1` was initialized as character `'0'`, which has ASCII code 48. Adding 2 results in 50, which corresponds to the ASCII character `'2'`.

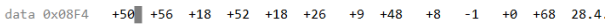


Figure 3: showing value of ch1 and ch2

Q5. Where are global variables (`g` and `f`) stored? Where are local variables (`x` and `y`) stored?

Global variables are stored in static data memory (SRAM) at fixed addresses. Local variables are stored on the stack in SRAM, allocated when the function is called.

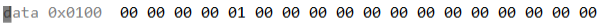


Figure 4: Global variable f

Globals → low SRAM addresses

Locals → higher SRAM addresses near the stack pointer : similar to figure 3 and 2

Q6. Explain why shift operation is preferred when doing division by 2.

Division by 2 using the division operator requires more CPU cycles (26 cycles). The bitwise shift right operator (`>>1`) is faster (23 cycles) because it directly manipulates bits. Shifts are more efficient in embedded systems where performance matters.

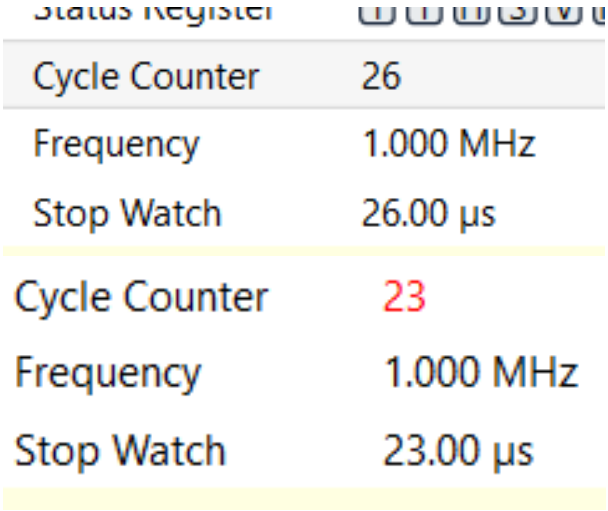


Figure 5: division and shift

Q7. In this application, the data memory usage is 6 bytes. Can you explain why it is 6 bytes?

The program defines several variables (`int g`, `float f`).Where int needs 2 bytes and float needs 4 bytes in total they occupy 6 bytes and both are global variable. Together, the compiler allocated 6 bytes total for active variables in this program.

Q8. How many instructions are used to realize the calculations for variable y (with division and shift operation), respectively?

The division statement (`y = x / 2 + g++`) requires more instructions 19 and took 26 cycles. The shift statement (`y = (x >> 1) + g++`) requires fewer instructions 18, and took 23 cycles. This confirms that the shift operation is more efficient.

Conclusion

This lab demonstrated how C code is compiled into assembly instructions on the ATmega328P, how variables are stored in memory, and why efficient operations such as bitwise shifts are preferred in embedded programming.